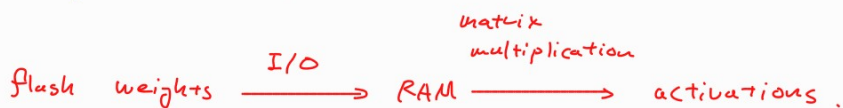


• edge device

$$M_{\text{weights}} > M_{\text{RAM}}$$

• weight offloading



• without

I/O compute
overlap

$$T_{\text{inference}} = \underbrace{T_{\text{I/O}}}_{\text{dominates}} + T_{\text{compute}} + T_{\text{management}}.$$

• streaming VLM

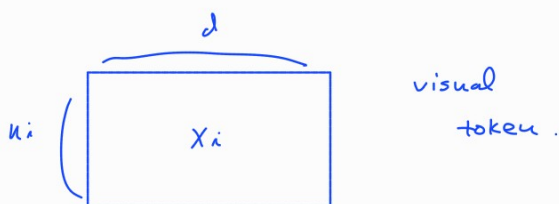
$$f_{\text{vision}}, f_{\text{proj}}, f_{\text{LLM}}$$

• visual token

from frame F_i

$$X_i = f_{\text{proj}}(f_{\text{vision}}(F_i)) \in \mathbb{R}^{n_i \times d}.$$

frame.



• inference stages

(i) prefill

init text prompt

init KV cache.

(ii) frame

appending

new frame's

visual token

extend KV cache.

(iii) decoding

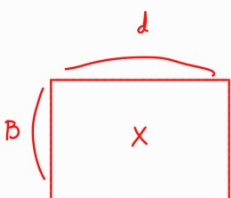
prev generated
token

extend KV cache.

& generate next token

• consider one attention head at one layer

• for B newly processed tokens



$$Q = XW_Q$$

$$K_{\text{new}} = XW_K$$

$$V_{\text{new}} = XW_V$$

append

$$K^+ = \begin{bmatrix} K_{\text{old}} \\ K_{\text{new}} \end{bmatrix}$$

$$V^+ = \begin{bmatrix} V_{\text{old}} \\ V_{\text{new}} \end{bmatrix}$$

Attention(X)

$$= \text{softmax} \left(\frac{Q(K^+)^T}{\sqrt{d_k}} + \textcircled{C} \right) V^+$$

casual mask.

permitted : 0

future : $-\infty$

영상을 보고 답하라 $\rightarrow$ 컷을 드는 영상 $\rightarrow$ 문제를 들었는가? $\rightarrow$ 컷을 들었습니다.

prompt

frames

user.

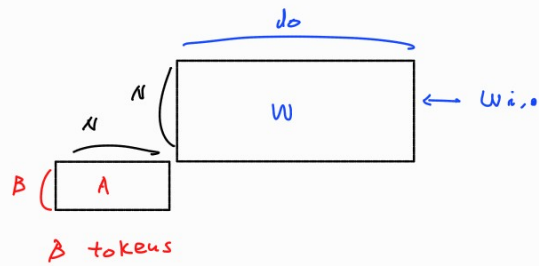
answer

(i) process multiple tokens together.

(ii) token outputs ignored.

process one token
at a time.

Write any :
projection

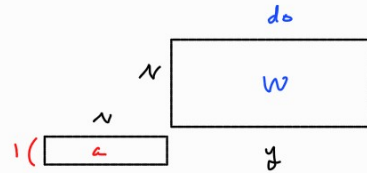


N : # of input channels

B : # of tokens processed together.

$W_{i,0}$: weight row associated with input channel i

• For one token

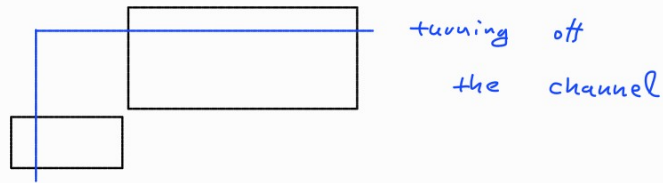


$$y = \sum_{i=0}^{N-1} a_i W_i$$

• with binary

mask $M \in \{0,1\}^N$

$$\tilde{y} = A \text{diag}(M) W \quad \xrightarrow{\text{reduce \# of rows to be loaded.}}$$



• surviving input channels

$$S = \{i : M_i = 1\}$$

$$|S| \leq R$$

• with sparsity ρ
no more than

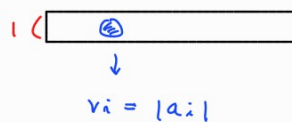
$$R = \lfloor (1-\rho) N \rfloor$$

↓
sparsity

• channel importance.

↓
($v_0, \dots, v_{N-1}$)

Single token.



multiple tokens



$$v_i = \frac{1}{B} \sum_b |A_{b,i}|$$

average of $|a_i|$'s

• how to choose M ? (input-dependent)

(i) top R : $\max_M \sum_{i=0}^{N-1} \underbrace{v_i}_{\text{importance}} M_i$

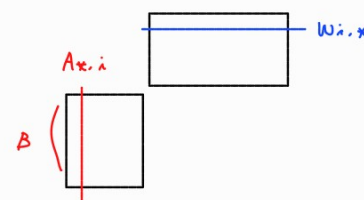
with (i) $M \in \{0,1\}^N$

(ii) $\sum_i M_i = R$

(iii) threshold : $M_i = \mathbb{1} [\underbrace{v_i}_{\text{importance}} \geq \underbrace{\tau}_{\text{threshold}}]$

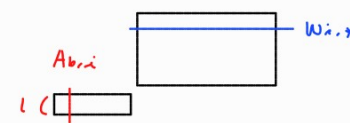
Activation magnitude is a proxy for importance
not an exact measure of accuracy.

• Approximation Error $Y - \tilde{Y} = \sum_i \underbrace{(1-\mu_i)}_{\text{turned off}} A_{x,i} W_{i,*} \in \mathbb{R}^{B \times d_o}$



• triangle inequality $\|Y_{b,*} - \tilde{Y}_{b,*}\|_2 \leq \sum_i (1-\mu_i) |A_{b,i}| \|W_{i,*}\|_2$

$\frac{1}{B} \sum_{b=1}^B \|Y_{b,*} - \tilde{Y}_{b,*}\|_2 \leq \sum_i (1-\mu_i) \underbrace{V_i}_{\text{expectation}} \|W_{i,*}\|_2$



make it less important!

Note. actual error may

depends on (i) weight norms ; task accuracy
(ii) cancellation must be measured
(iii) subseq layers separately.

Standard gated MLP

• $G = \text{SiLU}(X W_{\text{gate}})$ with $\text{SiLU}(x) = x \sigma(x)$
 $= x \cdot \frac{1}{1 + e^{-x}}$

• $U = X W_{\text{up}}$

• $H = G \odot U$

• $Y = H W_{\text{down}}$

projections $\left\{ \begin{array}{l} g, k, v \\ \text{gate \& up} \end{array} \right.$

shares input activations.

Modern VLMs has lesser sparsity.

(i) older ReLU-based LLMs : $\text{ReLU}(x) = \max(0, x) \Rightarrow \gg 0$'s
(ii) smooth activations : $\text{GeLU}(x) = x \Phi(x) \Rightarrow$ nonzero values
like SiLU or GeLU

Averaging over multiple visual tokens

suppresses token-specific fluctuations.

• with Z_b 's

$$\text{Var} \left(\frac{1}{B} \sum_b Z_b \right) = \frac{1}{B} \text{Var}(Z_i)$$

① independent.

② equal-variance.

↓
of
tokens

• actual visual tendency rather than

tokens are correlated : guaranteeing smoothing.

$$\text{Var} \left(\frac{1}{B} \sum_b Z_b \right) = \frac{1}{B^2} \left[\sum_b \text{Var}(Z_b) + 2 \sum_{b < c} \text{Cov}(Z_b, Z_c) \right]$$

$$= \frac{\sigma^2}{B} + \frac{2}{B^2} \cdot \frac{B(B-1)}{2} \cdot r \approx \frac{\sigma^2}{B} + r$$

• $CV(v) = \frac{\text{std}_i(v_i)}{\text{mean}_i(v_i)} \begin{cases} \text{small} & : \text{many important channels} \\ \text{large} & : \text{few important channels} \end{cases}$

• $CV_{VLM} < CV_{OPT-6.7B}$

$[1.07, 4.55]$ $[8.63, 11.65]$

↓
channels have similar importance :

① Good : choose less important neighbours $\Rightarrow IO \downarrow$

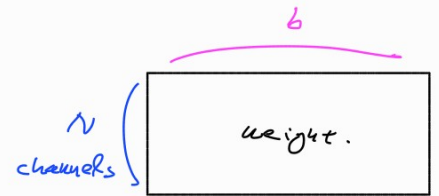
② Bad : retaining only : sparsity
tiny set of dominant channels versus > 90% (LTM)
40 ~ 60% (VLM)

flash throughput depends on read contiguity
= 순차적 읽기

(i) b : bytes per weight rows [byte]

(ii) $\beta(z)$: steady state throughput [second]

for reads of size z



• $\beta(z) \uparrow \beta_{max}$ as z increases.

small reads $\rightarrow$ overhead (by request processing)

large reads $\rightarrow$ bandwidth limit.

• with mask M : $D(M) = b \sum_i M_i$ $\leftarrow$ 요청이 하는 데이터.

$$\underbrace{L(M)}_{\text{latency [sec]}} = \frac{\underbrace{D(M)}_{\text{data [byte]}}}{\underbrace{\beta_{eff}(M)}_{\leftarrow \text{throughput, [byte/sec]}}}$$

• two access patterns

$$\text{latency } \frac{L_2}{L_1} > 1$$

even when data $\frac{D_2}{D_1} < 1$

if throughput $\frac{\beta_{eff,2}}{\beta_{eff,1}} \ll 1$

Note.

data $[D] = \text{byte}$.

throughput $[\beta] = \text{byte/second}$.

latency $[L] = \text{second}$.

$$\frac{\text{(latency)} L_2}{L_1} = \frac{\text{(data)} D_2}{D_1} \cdot \frac{\text{(throughput)} \beta_{eff,1}}{\beta_{eff,2}}$$

$$\frac{L_2}{L_1} > 1 \quad \text{iff} \quad \frac{D_2}{D_1} > \frac{\beta_{eff,2}}{\beta_{eff,1}}$$

chunk be max contiguous run of selected rows (=chunks)

- $\delta = \{1, 2, 4, 6, 7\} \Rightarrow h_m(1) = 1 \text{인 chunk}$ (즉, 1인 chunk)
- $C(m) = \{ \{1, 2\}, \{4\}, \{6, 7\} \}$ $h_m(2) = 2 \text{인 chunk}$ (즉, 2인 chunk)

- $h_m(r) = m$ 인덱스
즉, r인 chunk에 개수.

- # of chunks : $\sum_r h_m(r)$ $\rightarrow$ 즉, r인 chunk에 개수.

- # of surviving rows : $\sum_r r h_m(r)$ (= |S|)

Note. this hardware

- relevant structure discards

① exact positions

② distance between chunks.

proposed

latency model

$$\hat{L}(m) = \sum_{c \in C(m)} T(|c|) = \sum_r h_m(r) T(r) \quad [\text{sec}]$$

chunk chunk size

즉, r인 chunk 개수

$$\text{with } T(r) \approx \frac{L_{\text{profile}}(r, \beta)}{\beta} = \frac{r \cdot b}{\beta(r \cdot b)} \quad [\text{sec}]$$

- Note. • profiled, amortized cost
실행. 여러번 읽기에
공통 비용을 나눠 계산.

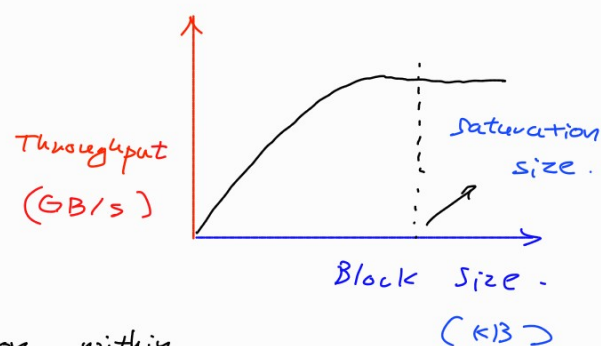
• latency of each

- Steady-state. (not isolated request)
읽기 요청이 계속 들어옴.

- concurrency already reflected.
동시 요청

offline profiling measures throughput by head size

- using
- ① Δ chunks
 - ② controlled placement.



- consider 128 MB dummy file. $\rightarrow$ completion within 20 min per device.
1 KB increments

- throughput: std is below 1% of the mean.

- Linux direct I/O & six thread C++ pool.

- saturation size: $z_{max} = \text{minimal } z$
with $\beta(z) \geq 0.99 \beta_{max}$. [GB/sec]

- $z_{max}^{AGX} = 236 \text{ KB}$, $z_{max}^{Nano} = 348 \text{ KB}$.

- $L_{actual}(\mu) \approx c \hat{L}(\mu)$ ($c > 0$)
measured estimated.

	AGX R^2	Nano R^2
LLVA-7B	.925	.950
NUCLT-2B	.929	.832

why proportional bias?

- chunk size & gap $\rightarrow$ fixed: profiling
mixed: inference.

- smaller workload $\rightarrow$ averaging
or weaker concurrency & prediction $\downarrow$

utility:
$$U_{actual}(c) = \frac{I(c)}{c \cdot T(c)} = \frac{1}{c} \cdot \underline{U_{estimated}(c)}$$

$\hookrightarrow$ almost: utility ordering
fixed unchanged

selecting strategy : $\max_M \frac{\sum_i v_i M_i}{\hat{L}(M)}$ [importance/sec]

with $M \in \{0, 1\}^N$; $0 < \sum_i M_i \leq R$

row-count budget.

• V masks $\rightarrow 2^N$ possibilities -

• instead rank chunks : $I(C) = \sum_{i \in C} v_i$; $U(C) = \frac{I(C)}{T(C)}$ $\rightarrow$ importance.
 $\rightarrow$ latency.

[importance/sec]

how to select $\leq R$ rows
 with $v = (v_0, \dots, v_{B-1})$

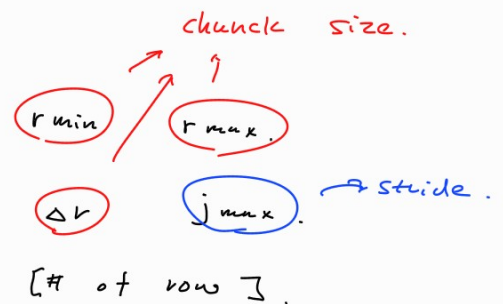
apply

$$\max\left(1, \left\lfloor \frac{b [KB]}{b [KB]} \right\rfloor\right)$$

① params : $\delta_{min}, \delta_{max},$

$\Delta\delta, j_{cap}$

[KB]



② candidate size : r_{min} to r_{max} , candidate chunk size.

slide windows

with stride : $\delta(r) = \min(r, j_{max})$

then utility $U_{i:r} = \frac{P_{i+r} - P_i}{T(r)}$

Note. prefix sum
 $I(i:r) = P_{i+r} - P_i$
 $= v_i + \dots + v_{i+r-1}$

③ sort candidate with decreasing utility

start with ; check (i, r) for : $M_{i:i+r} \leftarrow 1$; stop
 $M=0, k=0$ $M_{i:i+r} = 0$ $k+r \leq R$ $k \leftarrow k+L$ when
 $k=R$

repeat appending new chunk,

This is **greedy heuristic**.

- candidates $\Rightarrow$ miss benefits from
are evaluated individually. combining adjacent regions.

- restricted $\Rightarrow$ budget
candidate size. unused.

-
- maximizing a ratio under $|S| \leq R$
does not require filling R ; but we do.

- adjacent accepted
candidate may merge into maximal run

$S_{\min} \downarrow \Delta S \downarrow j_{\text{cap}} \downarrow \Rightarrow$ more candidate
& more overhead.

the paper tunes $\theta = (S_{\min}, j_{\text{cap}}) \rightarrow \Delta S = S_{\min}$
 $S_{\max} = Z_{\max}$.

- $S_{\min}, j_{\text{cap}} \rightarrow [0, 4, 8, \dots, 64] \text{ KB}$.
- tested separately for matrix shapes and devices.
- **10% sparsity**

-
- $T_{\text{selection}}(\theta) < 2 \text{ms}$

- prefer (i) fine-grained config
(ii) near feasible bounding,
with a small margin.

- each config $\rightarrow 30$ trials

- GPU Sorting accounts **80%** runtime

$\hookrightarrow$ data independent **Radix-Sort**; okay to use **random activation**

- final Δ_{min} : $8 \sim 40$ KB.
- Δ_{cap} : $8 \sim 36$ KB.
- larger matrix $\Rightarrow$ coarser settings.
- AGX supports more config than Nano.
- full sensitivity analysis remains future work.

Hot - Cold reordering.

- on calibration inputs

define
activation frequency $f_i = \frac{1}{c} \sum_{c=1}^c \mathbb{1} [i \in \text{Top } 50\% (v^{(c)})]$

input. (red arrow pointing to $v^{(c)}$)
importance (blue arrow pointing to $v^{(c)}$)

"probability of being one of top-50% important channels"

- sort channels by decreasing f_i ,
placing freq important channels together.

reorder $\left\{ \begin{array}{l} \text{weights} \\ \text{activation coordinates.} \end{array} \right. \Rightarrow$ dense computation preserved.

$$W' = PW, \quad A' = AP^T$$

permutation (red arrow pointing to P)

$$A'W' = AP^T P W = AW$$

- mean (AP^T) $\rightarrow 1.5 + 0.3$ ms for profiled down projection.
- activation - permutation overhead
- 95% upper confidence (red bracket under 0.3)

below 0.02% of total inference latency.

- f_i = historical frequency

- $U_i(x)$ = importance for current input.

⇒ reordering cannot determine.

best mask for future input.

- activation freq

analysis : $> 99\%$ ⇒ hot

$< 1\%$ ⇒ cold

⇒ many neurons lie

between extremes

∴ input dependence.

- layer-specific

: at 40% effective sparsity

sparsity also varies. one δ projection has 94% sparsity
